

Selection Sort

1 What is Selection Sort?

Selection Sort is a simple sorting algorithm.

The main idea is:

- Find the smallest element from the unsorted part.
- Place it at the beginning of the unsorted part.
- The sorted part grows one element at a time.
- Repeat until the complete array is sorted.

For ascending order:

Find minimum → Swap with first unsorted element

After every pass, one element is placed in its correct position.

2 Sorted and Unsorted Parts

Selection Sort divides the array into two parts:

1. **Sorted part**
2. **Unsorted part**

Initially:

Sorted part = Empty

Unsorted part = Entire array

After every pass, one element moves from the unsorted part to the sorted part.

For example:

$\underbrace{[]}_{\text{Sorted}} \quad \underbrace{[64, 25, 12, 22, 11]}_{\text{Unsorted}}$

After the first pass:

$\underbrace{[11]}_{\text{Sorted}} \quad \underbrace{[25, 12, 22, 64]}_{\text{Unsorted}}$

After the second pass:

$\underbrace{[11, 12]}_{\text{Sorted}} \quad \underbrace{[25, 22, 64]}_{\text{Unsorted}}$

And so on.

3 Selection Sort Algorithm

For an array of size n :

```
1 SelectionSort(A, n)
2
3 for i = 0 to n-2
4     min = i
5
6     for j = i+1 to n-1
7         if A[j] < A[min]
8             min = j
9
10    swap A[i] and A[min]
```

The variable min stores the index of the smallest element found in the unsorted part.

4 C++ Program

```
1 #include <iostream>
2 using namespace std;
3
4 void selectionSort(int A[], int n)
5 {
6     for(int i = 0; i < n - 1; i++)
7     {
8         int min = i;
9
10        for(int j = i + 1; j < n; j++)
11        {
12            if(A[j] < A[min])
13            {
14                min = j;
15            }
16        }
17
18        int temp = A[i];
19        A[i] = A[min];
20        A[min] = temp;
21    }
22 }
23
24 int main()
25 {
26     int A[] = {64, 25, 12, 22, 11};
27     int n = 5;
28
29     selectionSort(A, n);
30
31     cout << "Sorted Array: ";
32 }
```

```
33     for(int i = 0; i < n; i++)
34     {
35         cout << A[i] << " ";
36     }
37
38     return 0;
39 }
```

5 Time Complexity of Selection Sort

We analyze Selection Sort by counting the number of comparisons.

The basic operation is:

$$A[j] < A[\min]$$

This comparison is performed to find the minimum element.

5.1 Step 1: Outer Loop

The outer loop is:

```
1 for(int i = 0; i < n-1; i++)
```

Values of i are:

$$0, 1, 2, \dots, n-2$$

Therefore,

$$l = 0$$

and

$$u = n - 2$$

Using:

$$u - l + 1$$

we get:

$$(n - 2) - 0 + 1$$

$$= n - 1$$

Therefore, the outer loop executes:

$$n - 1 \text{ times}$$

So Selection Sort has:

$$n - 1 \text{ passes}$$

5.2 Step 2: Inner Loop

The inner loop is:

```
1 for(int j = i+1; j < n; j++)
```

For a fixed value of i :

$$j = i + 1, i + 2, \dots, n - 1$$

Therefore:

$$l = i + 1$$

and

$$u = n - 1$$

Using:

$$u - l + 1$$

we get:

$$(n - 1) - (i + 1) + 1$$

$$= n - i - 1$$

Therefore, for pass i , the number of comparisons is:

$$n - i - 1$$

6 Number of Comparisons in Each Pass

The number of comparisons decreases after every pass.

Pass	Comparisons
1	$n - 1$
2	$n - 2$
3	$n - 3$
$\vdots$	$\vdots$
$n - 1$	1

Therefore, total comparisons are:

$$(n - 1) + (n - 2) + (n - 3) + \dots + 2 + 1$$

7 Tracing Selection Sort

Let us take the following array:

$$A = [64, 25, 12, 22, 11]$$

Here:

$$n = 5$$

Therefore, the number of passes is:

$$n - 1 = 5 - 1 = 4$$

Total comparisons:

$$\frac{n(n-1)}{2} = \frac{5(4)}{2} = 10$$

So we expect:

4 passes and 10 comparisons

7.1 Pass 1

Initial array:

$$\underbrace{[]}_{\text{Sorted}} \quad \underbrace{[64, 25, 12, 22, 11]}_{\text{Unsorted}}$$

Find the smallest element from the unsorted part.

Comparisons:

$$64 \text{ vs } 25$$

Minimum becomes 25.

$$25 \text{ vs } 12$$

Minimum becomes 12.

$$12 \text{ vs } 22$$

Minimum remains 12.

$$12 \text{ vs } 11$$

Minimum becomes 11.

Swap 64 and 11.

After Pass 1:

$$\underbrace{[11]}_{\text{Sorted}} \quad \underbrace{[25, 12, 22, 64]}_{\text{Unsorted}}$$

Number of comparisons:

4

Cumulative comparisons:

4

7.2 Pass 2

Current array:

$\underbrace{[11]}_{\text{Sorted}} \quad \underbrace{[25, 12, 22, 64]}_{\text{Unsorted}}$

Find the smallest element from:

[25, 12, 22, 64]

Comparisons:

25 vs 12

Minimum becomes 12.

12 vs 22

Minimum remains 12.

12 vs 64

Minimum remains 12.

Swap 25 and 12.

After Pass 2:

$\boxed{\underbrace{[11, 12]}_{\text{Sorted}} \quad \underbrace{[25, 22, 64]}_{\text{Unsorted}}}$

Number of comparisons:

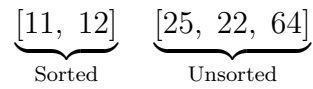
3

Cumulative comparisons:

$4 + 3 = 7$

7.3 Pass 3

Current array:



Find the smallest element from:

$[25, 22, 64]$

Comparisons:

25 vs 22

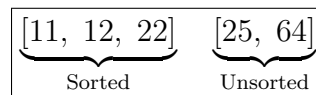
Minimum becomes 22.

22 vs 64

Minimum remains 22.

Swap 25 and 22.

After Pass 3:



Number of comparisons:

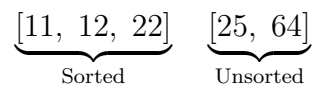
2

Cumulative comparisons:

$$7 + 2 = 9$$

7.4 Pass 4

Current array:



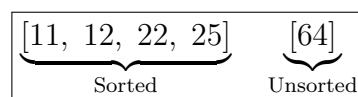
Only two elements remain.

Comparison:

25 vs 64

25 is already smaller.

After Pass 4:



Number of comparisons:

1

Cumulative comparisons:

$$9 + 1 = 10$$

Finally:

$[11, 12, 22, 25, 64]$ 

8 Complete Trace Table

For the array:

 $[64, 25, 12, 22, 11]$

the complete trace is:

Pass	Sorted Part	Unsorted Part	Comparisons
Initial	$[]$	$[64, 25, 12, 22, 11]$	0
1	$[11]$	$[25, 12, 22, 64]$	4
2	$[11, 12]$	$[25, 22, 64]$	3
3	$[11, 12, 22]$	$[25, 64]$	2
4	$[11, 12, 22, 25]$	$[64]$	1
Final	$[11, 12, 22, 25, 64]$	$[]$	10

9 Tracing in a Simple Way

The same trace can be remembered as:

 $[64, 25, 12, 22, 11]$
 $[11] \quad [25, 12, 22, 64]$
 $[11, 12] \quad [25, 22, 64]$
 $[11, 12, 22] \quad [25, 64]$
 $[11, 12, 22, 25] \quad [64]$
 $[11, 12, 22, 25, 64]$
The boxed portion represents the **sorted part**.The remaining portion represents the **unsorted part**.

10 Why Does the Number of Comparisons Decrease?

After every pass, one element is placed in its correct position.

Therefore, that element does not need to be considered again.

For example, for $n = 5$:

Pass	Unsorted Elements
1	5
2	4
3	3
4	2

Therefore, comparisons are:

$$4 + 3 + 2 + 1$$

$$= 10$$

11 Selection Sort: Quick Revision

Concept	Result
Number of passes	$n - 1$
Comparisons in pass i	$n - i - 1$
Total comparisons	$\frac{n(n-1)}{2}$
Highest-order term	n^2
Time Complexity	$O(n^2)$